OverTheWire

Bandit Assignment (AUV)

~Aditi

First all of, we need to connect to game's server.

For which I used **ssh bandit0@bandit.labs.overthewire.org -p 2220**

SSH	Invokes secure shell to connect to remote computers
bandit0	username
@bandit.labs.overthewire.org	Host address
-p 2220	Port 2220

The server then asks for a password that is bandit0. This logs you in as bandit0.

LEVEL 0:

The next password was stored in the readme file in the home directory. To access it, I used the following commands:

Ls: to list the files in directory (shows Readme).

Cat: To display the password inside Readme file.

Then I used exit command to logout, again ssh, and then entered the new password to login into bandit1.

LEVEL 1:

The password for next level was stored in a file named '-'(a single dash) in the home directory. Here we cannot directly use cat - as commands use dash to introduce options, rather than display text. So, we'll have to specify the path of the file instead of the name. So, I used **cat ./- (This tells the system to look in the current directory for a file named -).**

Again, I got the password to login into bandit2.

LEVEL 2:

The next password was stored in a file called “spaces in this filename” in the home directory.

I used **cat “spaces in this filename”.** Not using the (“”) quotation marks, the terminal reads for separate files.

Again, I got the password to login into bandit3.

LEVEL 3:

The next password was stored in a hidden file in the inhere directory.
First, I used **cd** command to go inside the inhere folder.

Hidden files don’t show up with regular **ls** (list) command, so I used **ls -a** (all).

The hidden file showed up with a dot in front of the file name. Then I again used **cat** command to rad the text inside it.

LEVEL 4:

- The next password was stored in the only human readable file in the **inhere** directory. When I used the **ls** command, there are 10 files named -file00 to -file09. First I used **cat** command in each file, only one contained the human readable text i.e the password.
- The alternative that I did was using the **file** command. **File command looks at the actual data structure to tell what the file is.** Since the filename starts with dash, again I used the **./** thing. **Using an (*), all the files can be checked at once.** Final command: **file ./***
  All the files except one displayed “**data”**. And -file07 showed “**ASCII Text”**. This was the required file, then I used cat command with this file and got the password for the next level.
- OR I could use **grep** command along with **file** command. **Grep** is used to search for text patterns in a file.

LEVEL 5:

The next password was stored in a file somewhere under the **inhere** directory and was human readable, 1033 bytes in size and not executable. To find this password, I directly used **find** command along with the requirements.

Final command: find . -type f -size 1033c ! -executable

- Find . – start searching in the current directory
- -type f – Look for files only, ignore folders
- -size 1033c – Look for files exactly 1033 bytes long.
- ! -execuable – here the **!** means not.

This command gave output as a single path, then I used **cat** command for this file path and got the password for next level.

LEVEL 6:

The password for the next level was stored somewhere on the server and was owned by user bandit7 and group bandit6 and was 33 bytes in size.

I used the **find** command to search file in the server.

Final command: **find / -user bandit7 -group bandit6 -size 33c 2>/dev/null**

- Find / -> Search starting from root of entire system.
- -user bandit7 -> Filter for files owned by bandit7
- -group bandit6 -> files belonging to group bandit6
- -size 33c -> 33 bytes exactly.
- 2>/dev/null -> It redirects all "**permission denied"** errors to a blackhole, so that only correct file path shows up.

This command returned one clear file path. Then I used **cat** command on that path to reveal the password.

LEVEL 7:

The password was stored in text file **data.txt** next to the word millionth. I used the **grep** command to search the file and print only the line containing the given word. The output gave the required password.

LEVEL 8:

The next password was stored in the file data.txt and was the only line of text that occurred once.

For this uniqueness, I used the **sort and uniq -u** command. Sort basically groups all identical lies together, and **uniq -u** prints only those lines that are completely unique.

LEVEL 9:

The next password was located in the same data.txt file. This password was one of few human readable string, preceded by several '=' characters.

I tried using **cat** command, but did not get the desired output, It is likely that the file is binary and so cat command gave garbage values.

So, To scan binary file, I used **strings** command along with **grep.** Strings command pulls out printable characters from binary file and grep function gives the strings with one of the '=' sign.

Final command: **strings data.txt | grep "=="**

This gave the password to move to level10.

LEVEL 10:

The next password was stored in the file data.txt, which contained base 64 encoded data. I used **base64 -d data.txt** command to read the password.

Base64 is a way of representing binary data in an ASCII string format. The -d flag tells the system to translate that string back into its original, readable password.